

手摸手之掌握架构师的编程规范

方法命名

1. 获取单个对象的方法用 `get` 作前缀

例如：查询单个用户 `getStudent`，按照 ID 查询单个用户 `getStudentById`。

2. 获取多个对象的方法用 `list` 作前缀

例如：按照 IDS 查询多个用户，`listStudentByIds`。

3. 获取统计值的方法用 `count` 作前缀

例如：统计全量用户，`countUser`。

4. 插入的方法用 `save` 作前缀

例如：新增用户，`saveUser`。

5. 删除的方法用 `remove` 作前缀

例如：删除用户，`removeUser`。

6. 修改的方法用 `update` 作前缀

例如：修改用户，`updateUser`。

领域模型命名规约

1. 数据对象

`xxxDO`，`xxx` 即为数据表名。

2. 数据传输对象

xxxDTO, xxx 为业务领域相关的名称。

3. 展示对象

xxxVO, xxx 一般为网页名称。

4. 注意事项

POJO 是 DO/DTO/BO/VO 的统称, 禁止命名成 xxxPOJO。

API 路径规约

1. Get 方法尽量把 ID 等变量放到路径上。

例如: 获取指定用户的信息。

```
/user/{id}
```

2. 多个不可分割的单词, 使用中划线拼接。

例如: 用户验证码接口。

```
/user/verify-code
```

3. 参数使用驼峰拼写。

例如: 获取指定用户购买的指定商品。

```
/order/{productId}
```

4. 指向集合的复数名称。

例如: 获取所有用户列表接口。

```
/users
```

5. 不用使用动词定义 URL

错误示例：

```
/update/user
```

或者：

```
/get/user
```

正确应该通过 HTTP 方法的语义来定义 URL 的行为。

比如说获取用户：

```
GET  
/user/{id}
```

添加用户：

```
POST  
/user
```

修改用户：

```
PUT  
/user
```

6. 对非资源 URL 使用动词

如果有一个接口，并不是 CRUD 操作，这种情况可以使用动词。

例如：向用户发送邮件接口。

```
/user/{id}/send-mail
```

7. 在嵌套资源的 URL 中使用关系

获取指定订单中所有商品列表。

```
GET /order/{id}/products
```

获取指定订单中所有指定商品信息。

```
GET /order/{orderId}/product/{productId}
```

注释规范

1. 注释说明意图即可，无需补充冗余字段

【强制】Class、Interface、Enum、@interface 等文件类型，类上注释仅需说明类的意图即可。不需要补充时间和创建人，因为 往往开发代码的不止是一个人，容易造成信息干扰。需要的话，查看提交记录即可。

```
/**
 * 适配第三方框架的线程池
 */
public interface ThreadPoolAdapter {

}
```

2. 方法上需要添加注释

【强制】方法上需添加注释，并说明清楚方法的意图（接口实现类无需注释）；必要时描述 @param @return 。

```
/**
 * 适配第三方框架的线程池
 */
```

```
public interface ThreadPoolAdapter {  
  
    /**  
     * 修改框架线程池的核心参数  
     *  
     * @param threadPoolBaseInfo 修改线程池的基础参数  
     * @return 线程池核心参数修改结果  
     */  
    boolean updateThreadPool(ThreadPoolBaseInfo threadPoolBaseInfo);  
}
```

如果方法为内部引用方法，并且方法名称见名知意，无需方法注释。

3. 方法块内部注释规范

【强制】方法内部的注释，应该新起一行，而不是跟在代码后面。

正例：

```
// 刷新动态线程池参数  
refreshDynamicPool(parameter, executor);
```

反例：

```
refreshDynamicPool(parameter, executor); // 刷新动态线程池参数
```

4. 方法命名说明方法本身意图

【强制】私有方法尽量通过方法命名说明方法语义。

代码开发规约

【强制】类、方法和变量的命名要做到顾名思义，避免使用缩写。

【强制】静态变量使用大写，多个单词使用下划线连接。示例：

MESSAGE_CENTER_SEND_TYPR。

【强制】捕获的异常名称命名为 ex；捕获异常且不做任何事情，异常名称命名为 ignored。

【强制】返回值变量使用 result 命名；循环中使用 each 命名循环变量；map 中使用 entry 代替 each。

result 命名示范：

```
private void parseDate(String data) {  
    Result result = JSONUtil.parseObject(data, Result.class);  
    return result;  
}
```

或采用 result 为前缀：

```
private void parseDate(String data) {  
    Result resultDate = JSONUtil.parseObject(data, Result.class);  
    return resultDate;  
}
```

each 命名示范：

```
appNameLeaseMap.values().forEach(each -> appNameLeaseList.add(each));
```

或是 for 循环：

```
for (Lease<InstanceInfo> each : appNameLeaseMap.values()) {  
    appNameLeaseList.add(each);  
}
```

【强制】业务系统中优先使用 Guava、HuTool、Common3 等工具类中的方法，不存在指定方法时再创建自定义工具类，禁止创建相同语义方法的工具类。

备注：定义组件项目时，尽量使用自定义工具类，避免因版本问题导致不确定的异常。

【强制】空实现接口或类不允许存在空格。

// 正例

```
public interface AdapterThreadPoolMonitor extends ThreadPoolMonitor {  
}
```

// 反例

```
public interface AdapterThreadPoolMonitor extends ThreadPoolMonitor {  
  
}
```

【建议】优先使用卫语句替换嵌套判断，提高代码简洁度。

【建议】方法参数自定义实体对象别名尽量使用 requestParam，非自定义对象可以使用对应语义命名。

```
@GetMapping("/add")
@ApiOperation(value = "新增购物车")
public Result<CartItemRespDTO> addCartItem(@RequestBody CartItemAddReqDTO requestParam) {
    return Results.success(null);
}
```



更新: 2023-07-17 17:46:26

原文: <<https://www.yuque.com/magestack/12306/bmatdhq46cegg2xe>>